



FastAPI – Routing & Request/Response

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence



Session Outline

1. Path Operations & HTTP Verbs

2. Path Parameters

3. Query Parameters

4. Request Bodies with Pydantic

5. Response Model & Status

7. Static Files & Templates

8. Strategy for DS & AI Students

9. Worked Example & Labs

10. Review & Summary



Learning Objectives

By the end of this session, you will be able to:

1. Declare path operations for every HTTP verb and map them to CRUD.
2. Read/validate **path parameters**; explain why route order matters.
3. Read/validate **query parameters**; constrain them with `Query()`.
4. Use a **Pydantic** `BaseModel` as a request body; explain the 422.
5. Shape the response with `response_model` and `status_code`.
6. Raise `HTTPException` for controlled errors.
7. Serve static files; know when a Jinja2 template fits.



Where We Are

- Week 6: a minimal FastAPI app runs.
- This week: make it *do* things – endpoints for every verb, input from path/query/body, shaped output, proper errors.
- Keep the **REST map** on the board: verb $\leftrightarrow$ CRUD action $\leftrightarrow$ status code.



Path Operations & HTTP Verbs

One Decorator + One Function

Decorator	Verb	CRUD	Success code
@app.get	GET	Read (retrieve)	200
@app.post	POST	Create	201
@app.put	PUT	Replace completely	200
@app.patch	PATCH	Modify partially	200
@app.delete	DELETE	Delete	204

- One **resource**, several paths: /items, /items/{id}.
- Resource names are **plural nouns**; the verb is the HTTP method, never in the URL (/getItem is RPC).



Path Parameters

Path Parameters

```
1 @app.get("/items/{item_id}")  
2 def read_item(item_id: int):  
3     return {"item_id": item_id}
```

- Type hint drives conversion + validation: `/items/5` $\rightarrow$ 5; `/items/abc` $\rightarrow$ 422, function never runs.
- Types: int, str, float, bool, Enum.
- **Route order matters:** declare `/items/me` *before* `/items/{item_id}` – first match wins.



Query Parameters

Query Parameters

```
1 from fastapi import Query
2
3 @app.get("/items")
4 def list_items(
5     skip: int = Query(0, ge=0),
6     limit: int = Query(10, ge=1, le=100),
7     q: str | None = Query(None, min_length=2),
8 ):
9     ...
```

- Scalar arg *not* in the path → query param (/items?skip=0&limit=10).
- A default makes it optional; str | None = None → may be absent.
- Query() adds constraints (ge, le, min_length...) + /docs metadata.
- Use query params for **optional** extras (paging, filtering) – not to identify the resource.



Request Bodies with Pydantic

A BaseModel Is the Request Body

```
1 from pydantic import BaseModel
2
3 class Item(BaseModel):
4     name: str
5     price: float
6     in_stock: bool = True      # default -> optional
7
8 @app.post("/items")
9 def create_item(item: Item):
10     return item
```

- BaseModel arg → FastAPI reads the JSON body, validates each field, builds the object.
- Missing/wrong-type field → 422 with a per-field error list.
- BaseModel $\approx$ @dataclass + runtime type checking + (de)serialisation.
- You can mix sources: path param + body + query param in one function.



Response Model & Status Codes

Shape What Comes Back

```
1 class ItemCreate(BaseModel):    # what the client sends
2     name: str; price: float
3
4 class ItemPublic(BaseModel):    # what the client may see
5     id: int; name: str; price: float
6
7 @app.post("/items", response_model=ItemPublic, status_code=201)
8 def create_item(item: ItemCreate):
9     saved = {"id": 1, "name": item.name,
10             "price": item.price, "secret": "xyz"}
11     return saved                # "secret" dropped: not in ItemPublic
```

- `response_model`: filters/formats output to a public schema; hides extra fields; documents /docs.
- `status_code`: 201 create, 204 delete, 200 otherwise. Separate input/output models.



Controlled Errors



HTTPException – Don't Crash, Don't Return None

```
1 from fastapi import HTTPException
2
3 @app.get("/items/{item_id}", response_model=ItemPublic)
4 def get_item(item_id: int):
5     item = __find(item_id)
6     if item is None:
7         raise HTTPException(status_code=404,
8                             detail="Item not found")
9     return item
```

- Becomes a clean JSON response: {"detail": "Item not found"}, status 404.
- Common codes: 400, 401, 403, 404, 409. Full auth: Week 11.



Static Files & Templates

Serving Static Files & Templates

```
1 from fastapi.staticfiles import StaticFiles
2 app.mount("/static",
3           StaticFiles(directory="static", html=True),
4           name="static")
5
6 from fastapi import Request
7 from fastapi.templating import Jinja2Templates
8 templates = Jinja2Templates(directory="templates")
9
10 @app.get("/items-page")
11 def items_page(request: Request):
12     return templates.TemplateResponse(
13         "list.html", {"request": request, "items": __all__()})
```

- StaticFiles: serve the project's frontend/ from the backend (no CORS in dev).
- Jinja2 = server-rendered HTML with `{{ }}` / `{% %}` – **optional** for an API + static-SPA project.



Strategy for DS & AI Students



Learning Strategy

- **Type hint = free validation.** You annotate a type; FastAPI does the checks, conversion, 422, and docs.
- **Pydantic is a familiar face:** BaseModel $\approx$ @dataclass $\approx$ “validate the schema before trusting the data”.
- **Keep the REST map on the board:** verb $\leftrightarrow$ CRUD $\leftrightarrow$ status code.
- **Refresh /docs after every change** – add a param/model, reload, try it live.



Worked Example & Labs

Worked Example: In-Memory items CRUD

```
1  _items: list[ItemPublic] = []
2  _next_id = 1
3
4  @app.get("/items", response_model=list[ItemPublic])
5  def list_items(skip: int = Query(0, ge=0),
6                limit: int = Query(10, ge=1, le=100)):
7      return _items[skip : skip + limit]
8
9  @app.post("/items", response_model=ItemPublic, status_code=201)
10 def create_item(data: ItemCreate):
11     global _next_id
12     it = ItemPublic(id=_next_id, name=data.name, price=data.price)
13     _items.append(it); _next_id += 1
14     return it
```

- GET /items/{id} and DELETE raise HTTPException(404) when id is unknown.

Lab 1: In-Memory CRUD

Hands-on practice

- GET /items, GET /items/{id}, POST, PUT /items/{id}, DELETE /items/{id}.
- Store items in a Python list.
- Return 404 via HTTPException when the id does not exist.

Lab 2: Validation & Response Shaping

Hands-on practice

- Add paging skip/limit with Query(..., ge=0) / ge=1, le=100.
- Define ItemCreate and ItemPublic as **separate** models.
- status_code=201 on POST, 204 on DELETE.
- Confirm on /docs that a bad body returns 422 with field-level detail.



Lab 3: Project Integration

Hands-on practice

- Write the basic routes for your project's main resource (list / detail / create / update / delete).
- Mount `StaticFiles` so the backend serves the `frontend/` folder.
- Open the frontend via the backend URL; confirm a `fetch()` to a new route works.



Review & Summary

Quick MCQ Review

1. `/items/{id}?limit=10` – which is path, which is query? id **path**, limit **query**
2. A BaseModel function argument is treated as? **The JSON request body, auto-validated**
3. Success code for creating a resource? **201 Created**
4. Body field of the wrong type →? **422 Unprocessable Entity**
5. Purpose of `response__model`? **Filter/format output to a public schema**
6. HTTPException is for? **A controlled error response (code + detail)**
7. Why `/items/me` before `/items/{id}`? **Routes match in order; else “me” becomes the id**



Self-Check Checklist

- ☐ Can declare path operations for all five verbs
- ☐ Know path vs. query parameters and route-order rule
- ☐ Can constrain params with `Query()`
- ☐ Can define a `BaseModel` body and explain the 422
- ☐ Use `response_model + status_code` deliberately
- ☐ Raise `HTTPException` instead of returning `None`
- ☐ Can mount `StaticFiles` for the frontend

Summary and Next Session

- **Path operation** = one decorator (get/post/put/patch/delete) + one path function; verb → CRUD + success code.
- **Path parameters** sit in the URL template, validated by type hint; declare fixed paths before parameterised ones.
- **Query parameters** are scalar args not in the path; a default makes them optional; Query() adds constraints.
- A **Pydantic** BaseModel argument = the request body; bad input → 422 with field-level errors.
- response_model shapes/sanitises output; status_code sets the success code; raise HTTPException for controlled errors.
- StaticFiles serves the frontend; Jinja2 templates are optional for an API + static-frontend project.
- Next: **Middleware, Dependencies & Sessions** – Depends(), middleware, cookies/sessions (Week 8).

Questions & Answers